Agentes RatatouilleIA

Como o Montador e o Avaliador se comportam em cinco experimentos simulados, e como o código funciona, passo a passo

Challenge 2 · Grupo CNV

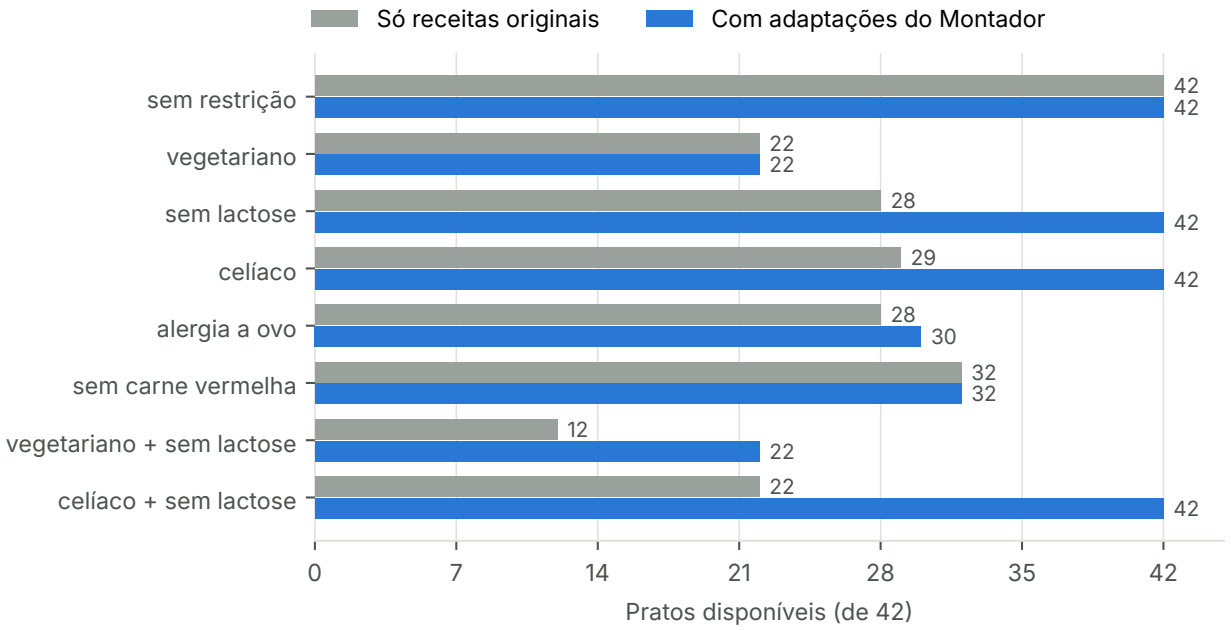
Base de conhecimento	50 receitas: 42 pratos e 8 acompanhamentos (12 do grupo, 25 do Ministério da Saúde, 13 geradas por IA)
Script	`scripts/experimentos.py`, que gera `resultados/experimentos.json`
Reprodutibilidade	semente aleatória fixa (42): rodar de novo dá os mesmos números

Nenhum experimento usa pessoas reais. São simulações para testar o comportamento dos agentes antes do teste com usuários. A seção 6 explica como cada um foi montado.

Limitações. São simulações, não testes com pessoas. O LLM do experimento 3 é simulado e não aprende, e os gostos do experimento 4 são inventados para testar o mecanismo. O teste com usuários reais continua sendo o passo que valida a hipótese do projeto.

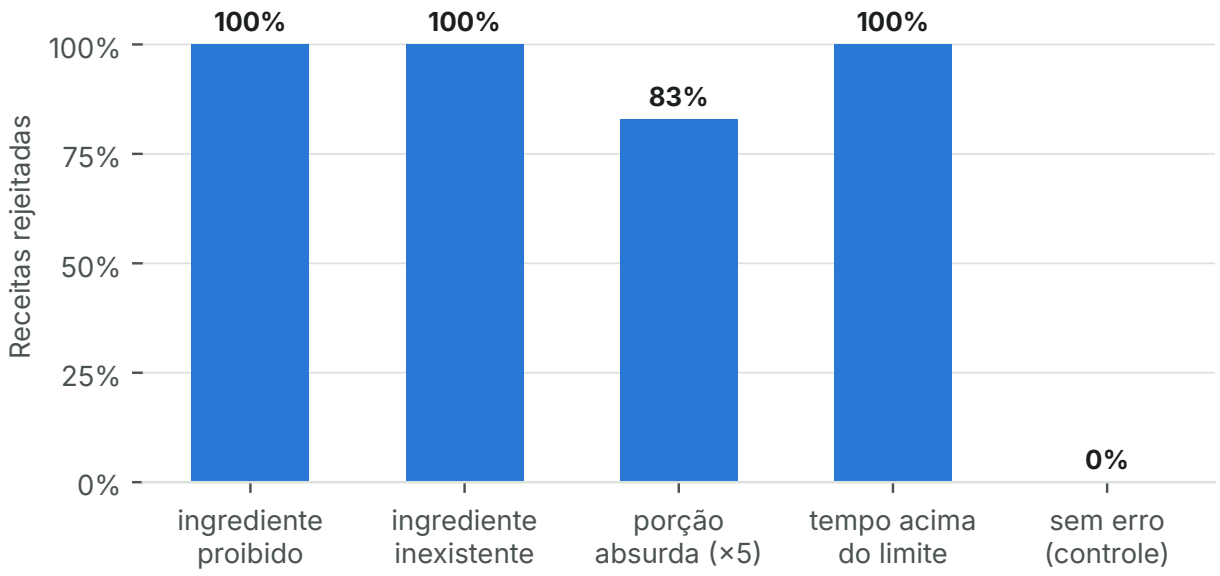
1 Quantos pratos cada restrição deixa disponíveis

As substituições do Montador recuperam muitos pratos que seriam perdidos: **sem lactose vai de 28 para 42** dos 42 pratos, celíaco de 29 para 42, e vegetariano sem lactose de 12 para 22. O vegetariano fica em 22 de propósito: o Montador não troca a proteína principal de um prato.



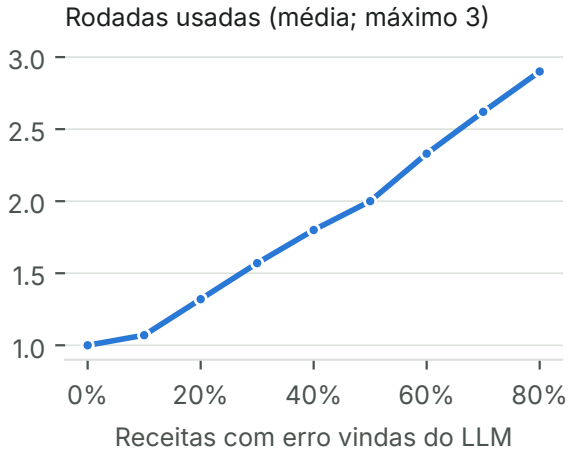
2 O Avaliador pega os erros que um LLM cometeria?

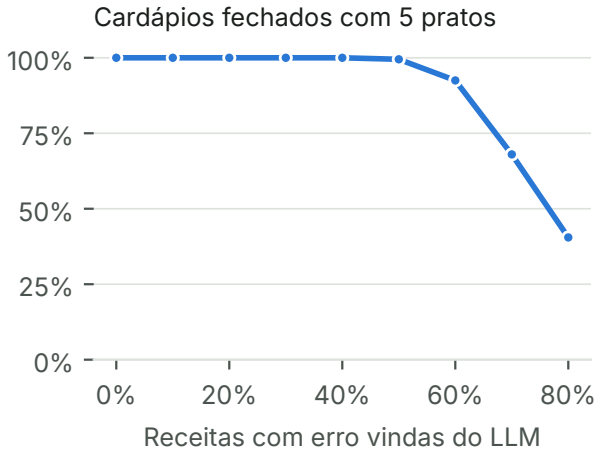
Com 300 receitas por tipo de erro, o Avaliador rejeitou **100% das receitas com ingrediente proibido, ingrediente inexistente ou tempo acima do limite**, e nenhuma receita do grupo de controle, que não tinha erro. A porção absurda escapou em 17% dos casos: quando o ingrediente multiplicado era pequeno, 5 vezes ele ainda ficava abaixo do limite por pessoa. Um limite de kcal por porção fecharia essa brecha.



3 Quantas rodadas até fechar o cardápio

Mesmo com um LLM simulado que não aprende com os motivos de rejeição, o laço de 3 rodadas **fecha o cardápio em 99,5% das vezes quando metade das receitas propostas vem com erro**, usando 2 rodadas em média. A partir de 70% de erro as 3 rodadas não bastam: 68% dos cardápios fecham com 70% de erro, e 40,5% com 80%.

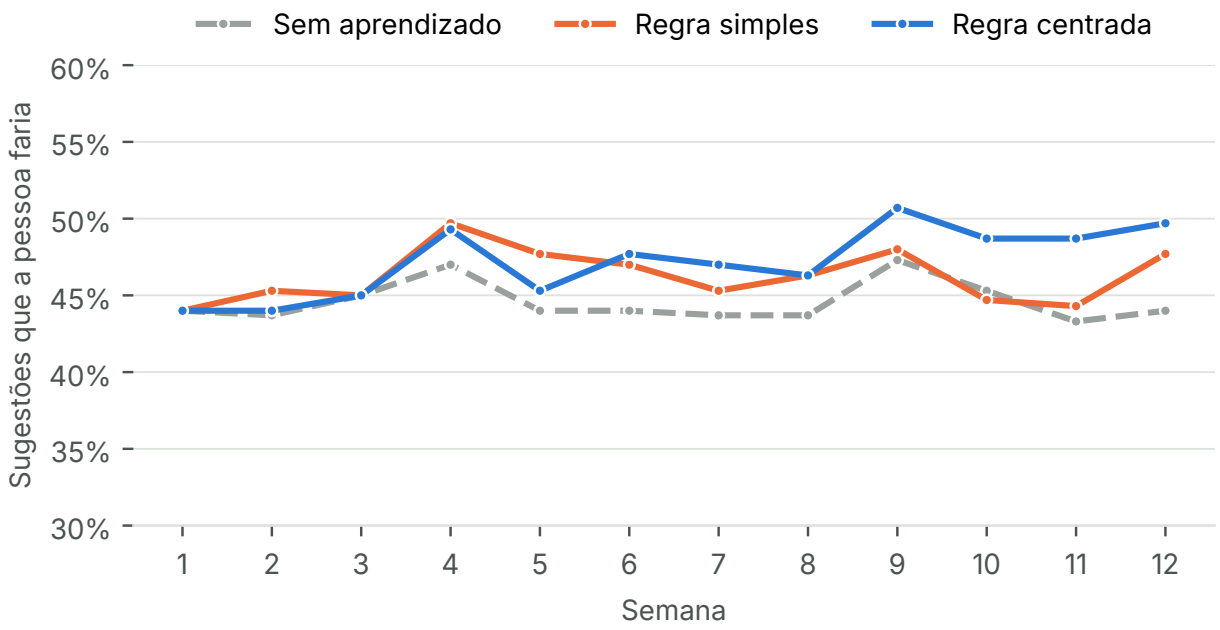




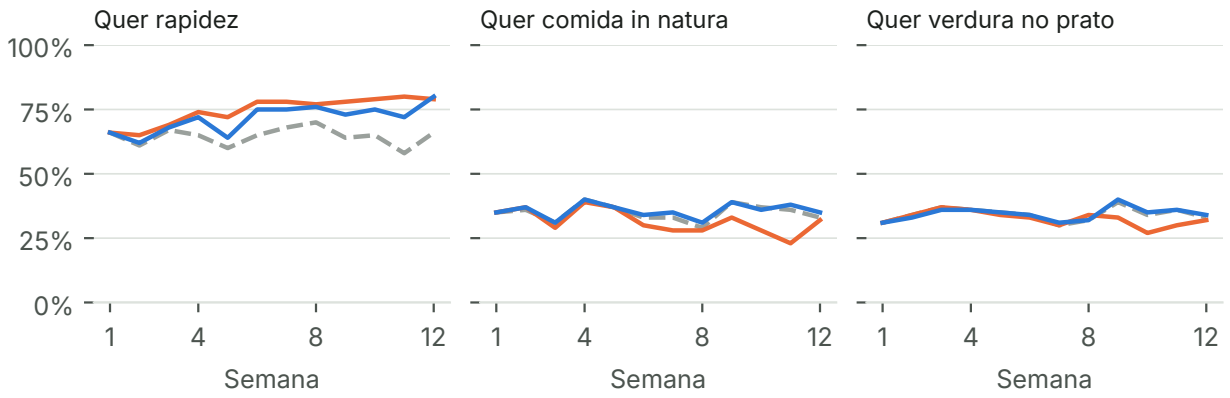
Receitas com erro	0%	10%	20%	30%	40%	50%	60%	70%	80%
Rodadas (média)	1,00	1,07	1,32	1,57	1,80	2,00	2,33	2,62	2,90
Cardápios completos	100%	100%	100%	100%	100%	99,5%	92,5%	68%	40,5%

4 A personalização melhora as sugestões com o tempo?

Com a regra centrada (`aprender_com_feedback`), a aceitação média sobe de **44% para 49,5%** nas últimas 4 semanas, contra 45% sem aprendizado. O ganho vem quase todo de quem quer rapidez (66% → 75%). A regra simples, que existia antes, chega a piorar o caso "comida in natura" (35% → 29%).



Separando por tipo de usuário simulado:

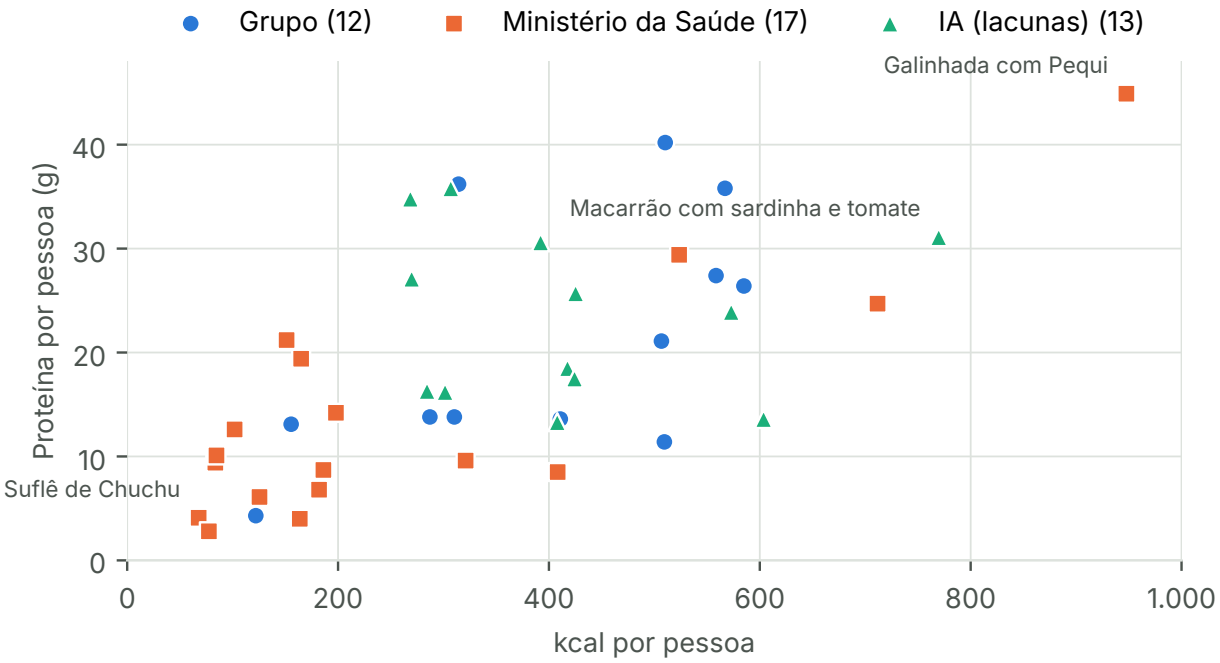


Aceitação nas últimas 4 semanas (semana 1 = 66%, 35% e 31%)	Sem aprendizado	Regra simples	Regra centrada
Quer rapidez	63,2%	79,0%	75,0%
Quer comida in natura	36,2%	29,0%	37,0%
Quer verdura no prato	35,5%	30,5%	36,2%
Todos	45,0%	46,2%	49,5%

Por que as curvas de "in natura" e "verdura" ficam planas. Quase todas as receitas já pontuam alto nessas duas características, então elas não ajudam a distinguir uma receita da outra, e o sistema não tem o que aprender. A regra centrada compara cada receita com a média das sugestões e por isso não piora esses casos, mas também não os melhora. O próximo passo são características mais finas, como a proporção de hortaliça no prato e a presença de ingredientes favoritos.

5 Calorias e proteína por pessoa em cada prato

Os pratos do Ministério são mais leves: **mediana de 165 kcal e 9,6 g de proteína por pessoa**, contra 459 kcal no grupo e 408 kcal nas receitas de IA. Muitos deles, como suflês e almôndegas, foram pensados para ir com arroz e feijão. Isso indica o próximo passo do planejador: montar a refeição como prato + acompanhamento.



Fonte da receita	Pratos	kcal por pessoa (mediana)	Proteína por pessoa (mediana)
Grupo	12	459	17,5 g
Ministério da Saúde	17	165	9,6 g
IA (lacunas)	13	408	23,9 g

Valores calculados pela TACO a partir dos ingredientes crus.

6 Como os experimentos foram simulados

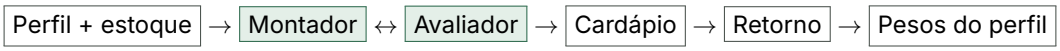
- **Experimento 1.** Para cada combinação de restrições, conta os pratos sem nenhum ingrediente proibido e os que o Montador consegue adaptar trocando ou removendo ingredientes.
- **Experimento 2.** Pega pratos válidos para um perfil sorteado, injeta um erro de cada tipo (300 receitas por tipo) e conta quantas o Avaliador rejeita. O grupo de controle não recebe erro.
- **Experimento 3.** Um "LLM" de mentira devolve receitas reais da base, cada uma com um erro com probabilidade p. Ele não aprende com os motivos de rejeição, então é o pior caso. São 200 simulações por taxa de erro, pedindo 5 pratos com no máximo 3 rodadas.
- **Experimento 4.** 60 usuários com um gosto escondido: querem rapidez, comida in natura ou verdura no prato. A pessoa "faz" uma sugestão se ela estiver entre as 20% melhores para o gosto dela. São 12 semanas com 5 sugestões por semana, comparando três versões: sem aprendizado, regra simples e regra centrada.
- **Experimento 5.** Nutrientes por pessoa de cada prato, somando os valores da TACO de cada ingrediente e dividindo pelas porções.

7 Como o código funciona, passo a passo

O sistema tem duas fases. Primeiro, um script monta a base de conhecimento a partir das fontes. Depois, a cada pedido do usuário, os agentes trabalham em cima dessa base: o Montador propõe receitas, o Avaliador verifica e o laço de rodadas junta os dois.

Arquivo	O que faz
`scripts/construir_base.py`	lê as fontes e gera a base em `kb/`
`scripts/medidas.py`	converte medidas caseiras em gramas
`agentes.py`	perfil, quadro compartilhado e Guardião (e o protótipo inicial)
`montador.py`	Agente Montador: propõe receitas
`avaliador.py`	Agente Avaliador: valida, pontua e calcula nutrientes
`planejamento.py`	laço de rodadas e aprendizado com o retorno
`scripts/gerar_receitas_llm.py`	gera receitas de IA para lacunas, com curadoria
`scripts/experimentos.py`	os experimentos deste relatório

O caminho de um pedido:



O Avaliador devolve ao Montador as receitas rejeitadas com o motivo, e os dois consultam a mesma base de conhecimento.

Passo 1. Montar a base de conhecimento

A base não é editada à mão: o script `construir_base.py` refaz tudo a partir de `fontes/`, então qualquer correção fica registrada no Git.

1. Lê as receitas do Ministério da Saúde, as do grupo e as geradas por IA.
2. Traduz cada ingrediente escrito (“cebola picada em cubos pequenos”) para um id da base (`cebola`), por uma lista de trechos de texto. Temperos como sal e cheiro-verde são ignorados.
3. Converte a medida caseira em gramas com `medidas.py`: volume da medida $\times$ densidade do ingrediente, ou número de unidades $\times$ peso médio da unidade.
4. Divide pelo rendimento da receita e grava tudo para 2 porções, o padrão do sistema.
5. Liga cada ingrediente à TACO para ter kcal, proteína, fibra e sódio por 100 g.
6. Calcula as tags de restrição de cada receita a partir dos ingredientes.
7. Recusa terminar se alguma receita usar um ingrediente sem cadastro, e escreve o relatório da base em `kb/`.

`scripts/medidas.py`

```
for medida, ml in VOLUME_ML.items():        # "xicara": 240, "colher de sopa": 15...
    if medida in t:
        estado = next((d for k, d in ESTADO.items() if k in ing), None)
        dens = estado if estado is not None else DENSIDADE.get(iid)
        ...
        return q * ml * dens, ""            # ½ xícara de óleo ≈ 108 g
```

Passo 2. Perfil e Guardião

O `Perfil` guarda o que vale para a pessoa: restrições (regras rígidas), tempo máximo, quantas pessoas comem, gostos e os **pesos** que dizem o que importa mais para ela. O Guardião transforma as restrições num conjunto de ingredientes proibidos, consultando a tabela `restricoes.csv`.

agentes.py

```
def proibidos(self, perfil: Perfil) -> set:
    out = set()
    for r in perfil.restricoes:              # ex.: ["vegetariano", "sem_lactose"]
        out |= self.restricoes.get(r, set()) # junta os proibidos de cada uma
    return out
```

É uma consulta a uma tabela, não uma decisão do LLM. Por isso a mesma restrição sempre bloqueia os mesmos ingredientes.

Passo 3. Agente Montador: propor receitas

O Montador só propõe; quem decide é o Avaliador. Ele tem dois modos com a mesma saída (uma lista de receitas em JSON), e cai sozinho no modo base se o LLM falhar.

montador.py

```
def propor(self, perfil, estoque, feedback, n=6):
    if self.modo == "llm":
        try:
            receitas = self._propor_llm(perfil, estoque, feedback, n)
            if receitas:
                return receitas
        except Exception as e:               # API fora, JSON quebrado etc.
            self.log.append(f"falha no LLM ({type(e).__name__}); usando a base")
    return self._propor_base(perfil, estoque, feedback, n)
```

Modo LLM. O prompt leva a lista de ingredientes permitidos (os proibidos já saíram), o tempo máximo, o estoque, dois exemplos de receita em JSON e **todos os motivos de rejeição acumulados**. Da resposta, o código recorta só o array JSON e descarta receitas fora do formato.

Modo base. Percorre as receitas da base e adapta cada uma ao perfil:

```
proteinas = {i: g for i, g in r["itens"].items()
             if self.ing[i]["categoria"] == "proteina"}
if proteinas and max(proteinas, key=proteinas.get) in evitar:
    return None      # "frango ensopado" sem frango não serve: descarta
...
sub = next((s for s in self.subs.get(i, [])
            if s == "remover" or (s not in evitar and s in self.ing)), None)
```

1. Pula acompanhamentos e receitas mais demoradas que o tempo do perfil.
2. Se a proteína principal for proibida, descarta a receita.
3. Troca os outros ingredientes proibidos pelo primeiro substituto permitido da tabela (leite → bebida vegetal) ou os remove (bacon, paio). Se não houver saída, descarta.
4. Ordena pelo aproveitamento do estoque, com uma pequena penalidade por troca, para preferir receitas originais.

Passo 4. Agente Avaliador: verificar e pontuar

O Avaliador não confia no Montador. Faz as checagens na ordem, da mais barata para a mais cara, e só dá nota a quem passa em todas.

avaliador.py

```
for checagem in (self._formato, self._ingredientes_existem):
    motivos = checagem(r)
    if motivos:
        return Resultado(r, False, motivos=motivos)
motivos = self._restricoes(r, perfil) + self._quantidades(r)
if r["tempo_min"] > perfil.tempo_max_min:
    motivos.append(f"demora {r['tempo_min']} min; o máximo do perfil é {perfil.
    tempo_max_min}")
if motivos:
    return Resultado(r, False, motivos=motivos)

c = self._caracteristicas(r, perfil, estoque)     # 5 valores entre 0 e 1
nota = sum(pesos[k] * v for k, v in c.items())     # soma ponderada pelos pesos
```

- **Checagens:** formato do JSON, ingredientes existentes na base, restrições (veto), porção por pessoa dentro do limite da categoria e tempo.
- **Características da nota:** quanto é comida in natura (descontando ultraprocessados), se tem verdura ou fruta, gosto da pessoa, rapidez e aproveitamento do que vai estragar.
- **Explicação:** as duas características que mais somaram viram a frase (“tem verdura ou fruta, é rápida”).
- **Nutrientes:** soma os valores da TACO de cada ingrediente e divide pelas porções.

Passo 5. O laço entre os dois agentes

A função `planejar` repete até 3 rodadas. Em cada uma, pede ao Montador só o que falta, o Avaliador julga, e os motivos de rejeição **se acumulam**, para o Montador não corrigir um erro e reintroduzir outro.

planejamento.py

```
for rodada in range(1, max_rodadas + 1):
    faltam = n - len(aprovadas)
    propostas = montador.propor(perfil, estoque, feedback, n=faltam + 2)
    ok, motivos, todos = avaliador.avaliar_lote(propostas, perfil, estoque)
    ...
    feedback["motivos"] = sorted(set(feedback["motivos"]) | set(motivos))  # acumula
    if len(aprovadas) >= n:
        break
```

Se não fechar em 3 rodadas, o status diz quantas receitas faltaram. A ideia é a interface mostrar as alternativas para a pessoa decidir, em vez de o sistema insistir.

Passo 6. Aprendizado com o retorno da semana

Depois da semana, a pessoa diz o que fez. A função compara cada característica com a **média das sugestões** e mexe só no peso do que diferencia o que foi feito do que não foi. O experimento 4 mostra por que isso é melhor que somar a característica direto.

planejamento.py

```
media = {k: sum(c[k] for c in caracteristicas.values()) / len(caracteristicas)
         for k in chaves}
for nome, c in caracteristicas.items():
    sinal = 1 if nome in feitas else -1
    for k in chaves:
        novo = perfil.pesos.get(k, 1.0) + taxa * sinal * (c[k] - media[k])
        perfil.pesos[k] = min(maximo, max(minimo, novo))   # entre 0,2 e 3
```

Os pesos ficam entre 0,2 e 3, para um retorno isolado não distorcer o perfil. Restrições nunca são aprendidas: só mudam quando a pessoa edita.

Passo 7. Receitas de IA com curadoria

`gerar_receitas_llm.py` pede ao LLM receitas para lacunas definidas: rápida para quem mora sozinho, vegetariana com leguminosa, marmita, baixo custo, sem lactose e sem glúten, nordestina do dia a dia. Cada uma passa pelo mesmo Avaliador, com as restrições da lacuna.

1. As aprovadas vão para `fontes/receitas_llm.csv` com `revisado_pelo_grupo=nao`.
2. As rejeitadas vão para `receitas_llm_rejeitadas.csv` com os motivos, para melhorar o prompt.
3. O grupo revisa, marca `sim` e roda `construir_base.py`. Antes da entrega, só as revisadas entram.

Passo 8. Como rodar

```
pip install -r requirements.txt && python scripts/construir_base.py
python demo_montador_avaliador.py     # Montador + Avaliador funcionando
python scripts/experimentos.py        # refaz os experimentos deste relatório
```